UPPSALA UNIVERSITET

Department of Information Technology

INSTRUCTIONS

Please check that you have the correct exam! This sheet should always be submitted, even if you haven't solved any of the exam problems.

Each solution should be written on a separate paper.

Write your exam code on each paper.

Please use only *one side* of the papers and do not use a pencil with red colour.

Sort the solutions in question order, with question 1 first, before you submit them.

FRONT SHEET FOR EXAMS	**DATE:**
Course name	
Your exam code	
Semester when you were first registered to the course:	Programme:
Time for submitting the exam:	Table number:

No.	**Solved problems (mark with X)**	**Points earned**
1		
2		
3		
4		
5		
6		
7		
8		
9		
10		
11		
12		
13		
14		
15		
16		
17		

Comments from the teacher

$\sum$

☐ Exam with bonus points: Grade is not shown.[1]

$5 \geq 61$ $4 \geq 48$ $3 \geq 36$

Exam grade:

1 The final result (points including bonus points and grade) can be found in Ladok after certification.

Advanced Computer Architecture, 10.0 c

Course code:1DT024
Semester: Spring 2024

Place: Bergsbrunnagatan 15: Sal 2
Time: 19th of August (08.00 - 13.00)
In case of an emergency you can call at 0734697380
Language: Answers in English (Swedish can be used if needed, but try to have the main points of your answer also in English)

General information

- There are 16 questions in this exam.
- The maximum score is 72p.
- You need 36p to pass (this corresponds to earning all 12 bonus points – each **bonus** question gives you 3p).
- You will automatically be given maximum points for the corresponding bonus questions if you have already received the corresponding bonus for the IRL, LAB, or HANDIN. You will not have to provide any answers for that question **but clearly state that for such a question you claim the bonus you have already received.**
- In case you think that you have earned a bonus point that is not accounted for, we suggest that you solve that question (just to be safe), but state in the solution that you think you should have received a bonus point. We will sort that out later…

Grades:
36p – 47p -> grade 3
48p – 60p -> grade 4
61p – 72p -> grade 5

Formatting your Anwers

Keep your answers simple.
Not readable and/or not understandable answers result in zero points.
If any of the questions seem unclear, **state your interpretation and assumptions clearly** and answer it.
Note that some of the questions have several sub questions (i.e. *What is 'X'* ***and*** *how does it improve "Y"...).*
Please answer all of them to get the full score.

Good luck!

//Stefanos, Pavlos, Shiming

1. (Bonus: Lab 1) Memory Optimization (3p)

Blocking can be used to improve the effectiveness of a cache.

a) Give a code example of how this technique is applied (first un-optimized and then optimized) **(2p)**
b) What single locality property (name the important one) is primarily improved using this scheme? **(1p)**

2. (Bonus: Handin 1) Caches (3p)

a) Make a drawing of a **physically addressed cache** with the following data

- Cache size: 2 Mbyte
- Cache line size: 64 byte
- Organization: 4-way
- CPU word size: 8 byte (this is the size of the data unit always delivered from the cache to the CPU)
- Physical address size: 42 bits (i.e., 4Tbyte of address space can be addressed)

Clearly show the address bits ranges used for the indexing, the comparisons, and multiplex ("mux") selects, etc. Describe, either in words or in logic, the functionality of all logic needed to resolve a read access to the cache.**(2p)**

b) Which bit ranges would change if the cache size was increased to 4Mbyte while all the other parameters stayed the same? State the new bit ranges (you do not need to make completely new drawing) **(1p)**

3. (Bonus: IRL 1) Caches (3p)

a) What is the purpose of the TLB in address translation? **(1p)**
b) Describe how a lookup is performed in a TLB (max 4 sentences). **(1p)**
c) What information is stored in a TLB entry? **(1p)**

4. (Bonus: Lab 2) Synchronization (3p)

a) What is the advantage of queue-based lock compared with a Test and Test&Set spinlock? **(1p)**
b) Write the pseudo-code for barrier synchronization, assuming that you have access to the synchronization primitives lock(L) and unlock(L). **(2p)**

5. (Bonus: Handin 2) Memory ordering (3p)

Dekker's algorithm is described by the following parallel pseudo-code:

<u>Initially:</u>

```
A:= 0;
B:= 0;
```

<u>Thread 0:</u>	<u>Thread1:</u>
A:= 1;	B:=1;
if (B==0) then	if (A==0) then
print("Thread 0 won");	print("Thread 1 won");

a) What feature is implemented by Dekker's algorithm? (I.e., what did Dekker use it for) **(1p)**
b) Under which of the three memory models: sequential consistency, total store ordering and/or weaker consistency, would it work as Dekker anticipated? **(1p)**
c) How would the code be changed in order to work for weaker models? **(1p)**

6. (Bonus: IRL 2) Coherence (3p)

Draw the two state transition diagrams (one for snooping the bus (*) and one for a CPU accessing the cache(**)) for an idealized MOSI cache coherence protocol where each state transition and output signal happens *"atomically"*. For each state transition arc between the coherence states, mark the input signal (what caused this transition)/output action (e.g., is Data provided by this cache?) **(3p)**

(*) Snooping transactions: ReadToShare (RTS), ReadToWrite (RTW), Invalidate (INV), Wrtiteback (WB).

(**) CPU accesses: CPUread, CPUwrite, CPUreplacement

7 (Bonus: Lab 3). Scalable MP (3p)

a) What is false sharing and under which of the parallel programming paradigms OpenMP, MPI and/or Pthreads, could an application typically experience such a problems? **(2p)**

b) How can you remove false sharing from an application? **(1p)**

8. (Bonus: Handin 3) Scalable MP (3p)

a) Draw the structure of a Dir_4B directory assuming that it tracks the coherence of 128 caches. Clearly show how many bits each field has (2p)
b) Describe the difference between coherence protocols based on a Dir_4B directory and a Dir_4NB directory? (1p)

9. (Bonus: IRL 3) In-Order Pipelines (3p)

A 2-way SMT pipeline and a 2-way super-scalar pipeline can both start the execution of two instructions each cycle. State how they differ in terms of:

a) How they select the instructions to execute. **(1p)**
b) What hardware they need to replicate compared with a 1-way implementation. **(1p)**
c) What kind of *parallelism* they exploit. **(1p)**

10. (Bonus: Lab 4) Parallelism (3p)

a) According to Flynn's taxonomy, what kind of parallelism is primarily explored with the SSE, MMX and AVX instructions of x86? Write the four-letter acronym and a brief explanation of how it works. (2p)
b) What is the other common parallelism expressed in Flynn's taxonomy and how does it differ from 10a? (1p)

11. (Bonus: Handin4) CPU and Pipelines (3p)

There are three different classes of hazards: Structural hazards, data hazards and control hazards.
Please state which of them (a single one) is <u>mainly</u> removed by:

a) Branch prediction? (1p)
b) Register renaming? (1p)
c) Superscalar architecture (compared with single-issue architectures)? (1p)

12. (Bonus: IRL 4) CPU and Pipelines (3p)

a) Explain the most fundamental technique used by GPUs to hide memory latency? **(1p)**
b) How many threads can a high-performing GPU typically execute at the same time today? (10s, 100s or 1000s) **(1p)**
c) Name two popular "high-level languages" that are commonly used to program GPUs today. **(1p)**

13. Cache Coherence (11p)

PART A (4p)

One of the simplest protocols for coherence is MSI (Modified, Shared, Invalid). It provides the basic functionality for the Single-Writer-Multiple-Reader Invariant in cache coherence. However, there are protocols with more states, for example, protocols that include an Exclusive "E" and/or Owned "O" state.

Keep your answers simple.

a. The Shared state "S" says nothing about how many other sharers there might be in the system. In contrast, an Exclusive "E" state implies that it is the only sharer in the system.
1) How do the M and E states differ?
2) How do S and E states differ?
3) From which of the MSI states (Modified, Shared, Invalid) do you reach the E state and with what BUS () and PROCESSOR () transactions? Draw two state diagrams, one for to show exactly these cases (do not bother with irrelevant states or transactions). If a state diagram is empty (no states), indicate it.
4) Why adding an Exclusive state to MSI, to form the MESI protocol, is useful? Give an example of a situation where a protocol with the E state (i.e., MESI) performs better than the basic MSI.

The Modified "M" state implies (among other things) that the cacheline is different than the data in memory. An Owned "O" state also implies that the cacheline is different than the data in memory.
1) How do the M and the O states differ?
2) How do the S and O states differ?
3) From which of the MSI states (Modified, Shared, Invalid) do you reach the O state and with what BUS (BUS_RTS, BUS_RTW, BUS_WB, BUS_INV) and CPU (CPUread, CPUwrire, CPUreplace) transactions? Draw two state diagrams, one for BUS transactions, one for CPU transactions, to show exactly these cases (do not bother showing irrelevant states or transactions). If a state diagram is empty

(no states), indicate it.
4) Why adding an Owned state to MSI, to form the MOSI protocol, is useful? Give an example of a situation where a protocol with the O state (i.e., MOSI) performs better than the basic MSI.

PART B (7p)
All the three RISC CPUs (e.g. cores in a multicore) in a MESI shared-memory sequentially consistent multiprocessor execute the following code.

CPU1	CPU2	CPU3
A = 1;	**while (A==0) {};**	**while (B==0) {};**
while (C ==0) {};	**B = 1;**	**C = A;**

The CPUs execute ALU instructions (register-to-register) and memory instructions, moving values between memory and registers. There are two kinds of memory instructions in this example: LD and ST
CPU3 will start slightly ahead of the others and will execute its first memory instruction before CPU2 starts its first memory instruction. CPU1 will start last. Then, the CPUs will take turns and execute one memory instruction each (this is an artificial constraint for exam purposes only).
Initially A, B and C reside only in memory and store the values 0. They belong to different cachelines.
After the parallel execution is done for all of the CPUs, the cachelines that are still in the caches will get replaced by data brought in by some other execution. This activity should also be reflected in your answer.

The following four bus transaction types can be seen on the snooping bus connecting the CPUs (cores):

- RTS: ReadtoShare (reading the data with the intention to read it)
- RTW, ReadToWrite (reading the data and requesting write permission)
- WB: Writing data back to memory
- INV: Invalidating other caches copies

Fill the solution in the Solution Sheet below (you can make a similar table on your own if that is more convenient, but keep exactly the same columns as in the Solution Sheer). Show every state change and/or value change of A, B and C in each CPU's cache according to one possible interleaving of the memory accesses. These actions should also be shown. For each line, also state what bus transaction occurs on the bus (if any) as well as which device is providing the corresponding data (if any).

(As a guide as to how you should fill out the table, this small example shows a parallel program where the following happens: CPU1 executes a load(A) instruction, CPU2 executes a load(B) instruction, CPU3 executes a load(B) instruction, CPU1 executes a store(A) instruction, and then CPU3, executes a store(B) instruction, eventually followed by the replacement actions to empty the caches. The data values stored at memory locations A and B are initially 1 and 0 respectively.)

CPU action	***Bus Transaction (if any)***	***State/value after the CPU action***						***Data is provided by*** *[CPU 1, 2, 3 or Mem] (if any)*
		CPU1		*CPU2*		*CPU3*		
		A	*B*	*A*	*B*	*A*	*B*	
Initially		*I*	*I*	*I*	*I*	*I*	*I*	
CPU1: LD A	*RTS(A)*	*E/1*						*Mem*
CPU2: LD B	*RTS(B)*				*E/0*			*Mem*
CPU3: LD B	*RTS(B)*				*S/0*		*S/0*	*CPU2*
CPU1: ST A		*M/0*						
CPU3: ST B	*INV(B)*				*I*		*M/1*	
...some time elapses .								
CPU1: replaces A	*WB(A)*	*I*						*CPU1*
CPU3: replaces B	*WB(B)*						*I*	*CPU3*

Solution Sheet

CPU action	**Bus Transaction (if any)**	**State/value after the CPU action**									Data is provided by [CPU 1, 2, 3 or Mem] (if any)
		CPU1			CPU2			CPU3			
		A	B	C	A	B	C	A	B	C	
Initially											

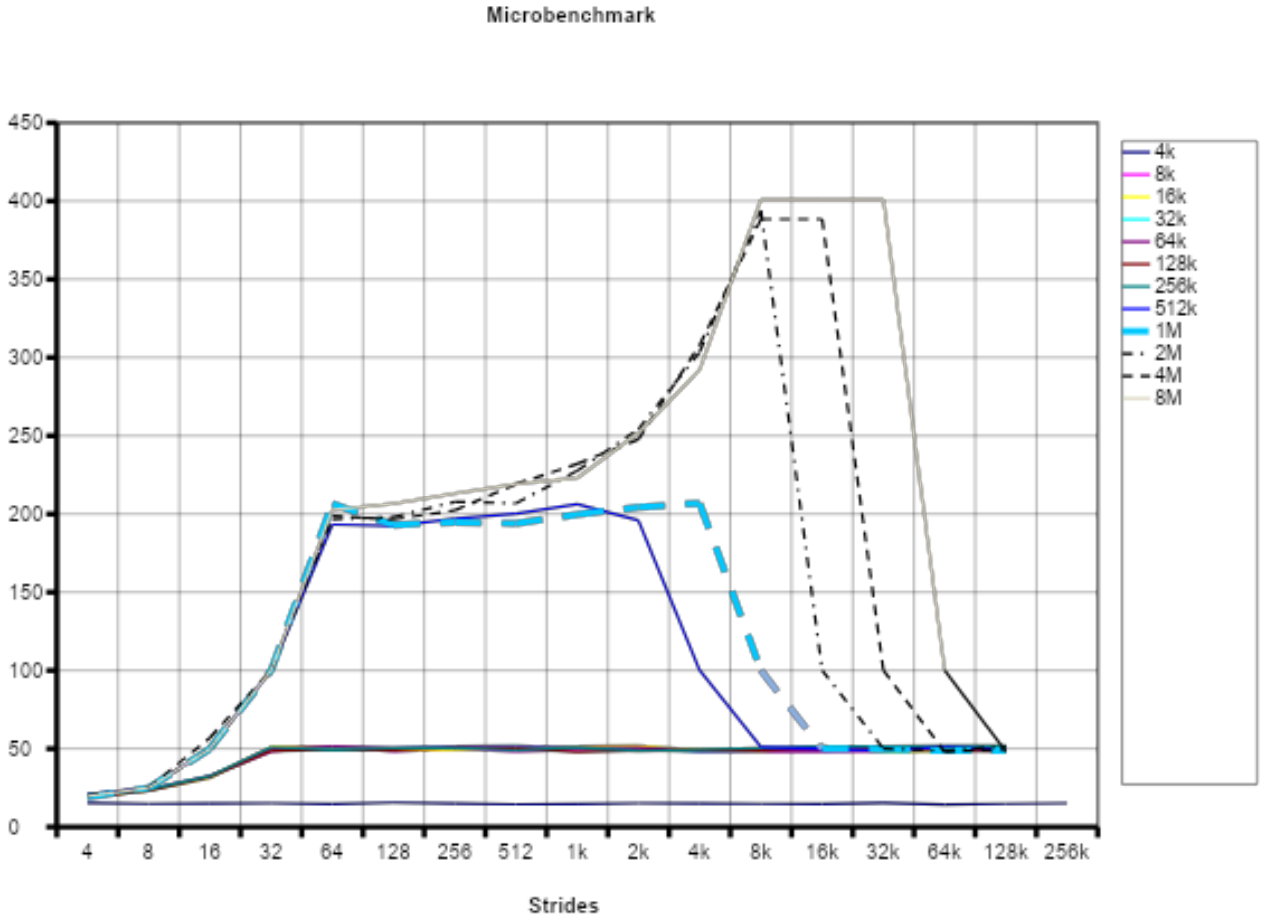
14. Microbenchmarks (8p)

A simple microbenchmark code below is used to draw curves that characterizes the properties of the underlying memory system. Initially, we assume that both cache and TLB are fully associative.

```
for (times = 0; times < Max; time++)          /* many times*/
  for (i=0; i < ArraySize; i = i + Stride)
    dummy = A[i];                             /* touch an item in the array */
```

The curve below shows the average access time to a memory system as a function of Stride for different ArraySizes.

a) **What is the cache line size and cache size for the 1st level cache? (2p)**
b) **What is the cache line size and cache size for the 2nd level cache? (2p)**
c) **What is the page size and how many entries does the TLB have? (2p)**
d) **How would the graph change if the TLB was made 8-way set-associative instead of fully associative? (2p)**



(*) The lowest curve contains only the 4k curve
Curves 8k – 256k look very similar
Curves 512k – 1M look similar between stride=64 and stride = 1k

15. Out of Order (9p)

Register Renaming (4p)

Assume the following code snippet:

```
double    v[2048];  //size(double)=8B
double    sum;
…
//We assume that this loop is repeated many times//
for (i =0, i < 2048, i++) {
   sum+=v[i];
}
```

Translated into pseudo-ASM :

```
LOOP:
1:  LDD   R4, 0(R1)       // R4 = v[i]
2:  ADDD   R6, R6, R4          // sum +=
3:  SUBI   R1, R1, #8     // i++
4:  BEQZ   R1, #LOOP      // last time?
```

Questions:

1. Show all loop dependencies. (1p)

2. Unroll the loop 3 times by applying register renaming. Also, show the mapping from Architectural to Physical registers. Assume that for the first iteration R1 is stored in Px and R6 in Py. (3p)

We provide you a reference start, you can continue from this:

```
1. LDD P1, 0(Px)
2. ADDD P2, Py, P1
3. SUBI P3, Px, #8
```

Rename Table (Initialized to the given reference start)
Feel in the boxes as you go in time with how each architectural register is renamed. If you like a different method to present the Rename Table feel free to show it otherwise.
Architectural Register to Physical Mapping (Time —>)

Arch. Regs.	Reference Start Initial Rename							
R1	Px							
R2								
R3								
R4	P1							
R5								
R6	Py							
...								

Out of Order Execution (5p)

Assume the following:

From	To	Latency
ALU	ALU	0 (can issue next cycle)
FP	ALU SD	2
LD	FP/INT ALU	3 "3 bubbles" (if L1 hit)

Branches: Adds 3 cycles, unless correctly predicted

64 bit architecture/application

L1: 32kB 4-way, CL = 32B, latency 3c

L2: 1MB 4-way, CL = 32B, latency 15c

Memory: Latency 200c

BP=2bit table, 3bit global branch history

Assume a dual issue Out-of-Order architecture.

1. Using your unrolled loop from above (Register Renaming - Question 2), show in which cycle each instruction is issued. (4p)

We provide you a reference start, you can continue from this:

Cycle	Instruction
1	`LDD P1, 0(Px)`
?	`ADDD P2, Py, P1`
1	`SUBI P3, Px, #8`

Since it is dual issue, on the first cycle both the first and third instruction can issue, but the second instruction cannot, since it has a Read-after-Write dependence with P1 of the first instruction.
Continue for the rest of the loop.

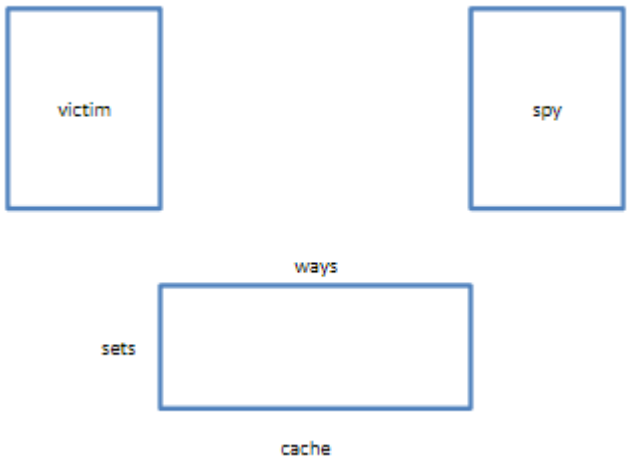
2. How many cycles does it take on average to complete a single loop? (1p)

16. Security (5p)

A cache side-channel leaks information via changes in the cache made by a victim program, changes that can be detected by an adversary.

1. What kind of program information can leak out of a cache side-channel from a non-speculative execution of a program? (1p)

2. How can an adversary extract this information from the cache? Hint: Think what the adversary has to do to detect specific changes in the cache --- assume that the adversary accesses the cache before and after the victim, but cannot directly read the victim's own cachelines in the cache. Use the diagram to help your description. (2p)



3. A speculative side-channel attack uses a cache side-channel in an entirely new way. The following code (in simple C) is a huge security threat in a core with speculative execution (branch prediction speculation) because it can directly leak the value of the variable "secret" via the cache side-channel. How does it do that? Detail you answer explaining step-by-step what happens in the core and in the caches. (Assume that cacheline size is 64 bytes.) (2p)

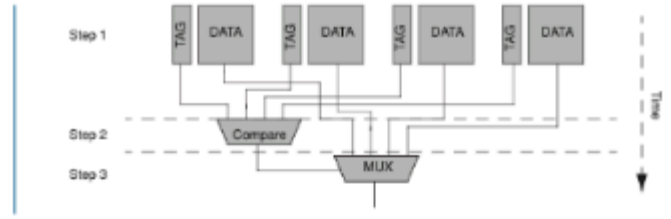
```
char x; // undefined
int array1_size = 100;
char array1[100]; // an array of 100 bytes: array1[0]…array1[99]
// the largest valid index for array1 is 99
char secret = 13; // a secret byte is located in memory just after the end
of array1
// and happens to map on exactly array1[100]
…
char array2[16384]; // a probe array
main(){
     …
     // attack
     x = 100;
     flush_cache; // clear the cache (and write back to memory its dirty
     contents)
     // assume that this is part of the attack
     if (x < array1_size)
          y = array2[ array1[x] * 64 ];
          …
     }
}
```

17. POWER (3p)

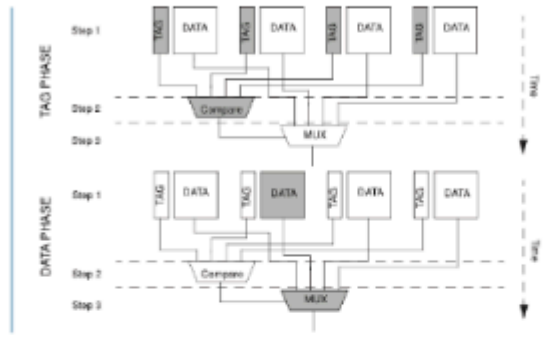
Explain (on the side of each with a few sentences) how the following techniques save power and how they affect performance (shaded components consume power):

For example, Normal Cache accesses all tags and data arrays at the same time. Best performance: quickest to deliver the data on a hit.
Explain the rest.

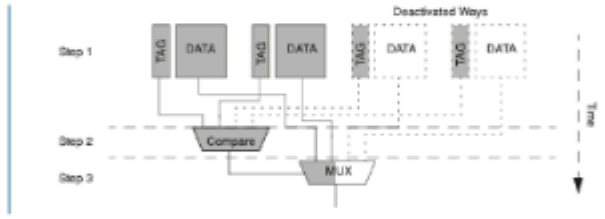
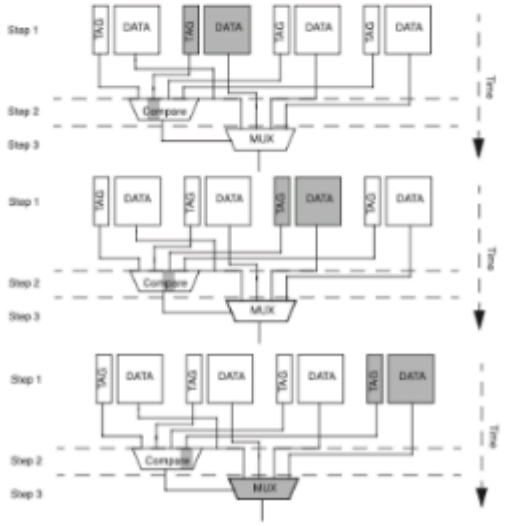
Normal cache



PHASED CACHE



CACHE RESIZING



SEQUENTIAL-ACCESS CACHE